20

Updating and Deleting Data

In this chapter, you will learn how to use the `UPDATE` and `DELETE` statements to further manipulate table data.

Updating Data

To update (modify) data in a table, you use the `UPDATE` statement. `UPDATE` can be used in two ways:

- To update specific rows in a table
- To update all rows in a table

Let's take a look at each of these uses.

Caution

Don't Omit the `WHERE` Clause Take special care when using `UPDATE` because it is very easy to mistakenly update every row in a table. Please read this entire section on `UPDATE` before using this statement.

Note

`UPDATE` and Security Use of the `UPDATE` statement can be restricted and controlled. See Chapter 28, "Managing Security."

The `UPDATE` statement is very easy to use—maybe even too easy. An `UPDATE` statement is made up of three parts:

- The table to be updated
- The column names and their new values
- The filter condition that determines which rows should be updated

Let's take a look at a simple example. Customer 10005 now has an email address, and his record needs to be updated. The following statement performs this update:

Input

```
UPDATE customers
SET cust_email = 'elmer@fudd.com'
WHERE cust_id = 10005;
```

The `UPDATE` statement always begins with the name of the table being updated. In this example, it is the `customers` table. The `SET` command is then used to assign the new value to a column. In this example, the `SET` clause sets the `cust_email` column to the specified value:

```
SET cust_email = 'elmer@fudd.com'
```

The `UPDATE` statement finishes with a `WHERE` clause that tells MySQL which row to update. Without a `WHERE` clause, MySQL would update all the rows in the `customers` table with this new email address—definitely not the desired effect.

Updating multiple columns requires slightly different syntax:

Input

```
UPDATE customers
SET cust_name = 'The Fudds',
    cust_email = 'elmer@fudd.com'
WHERE cust_id = 10005;
```

When updating multiple columns, only a single `SET` command is used, and each *`column = value`* pair is separated by a comma. (No comma is specified after the last column name.) In this example, columns `cust_name` and `cust_email` are both updated for customer 10005.

Tip

Use Subqueries in an `UPDATE` Statement You can use subqueries in `UPDATE` statements, which enables you to update columns with data retrieved using a `SELECT` statement. Refer to Chapter 14, "Working with Subqueries," for more information on subqueries and their uses.

Tip

The `IGNORE` Keyword If your `UPDATE` statement updates multiple rows and an error occurs while updating one or more of those rows, the entire `UPDATE` operation is cancelled (and any rows updated before the error occurred are restored to their original values). To continue processing updates, even if an error occurs, use the `IGNORE` keyword, like this:

```
UPDATE IGNORE customers ...
```

To delete a column's value, you can set it to NULL (assuming that the table is defined to allow NULL values). You can do this as follows:

Input

```
UPDATE customers
SET cust_email = NULL
WHERE cust_id = 10005;
```

Here the NULL keyword is used to save no value to the cust_email column.

Deleting Data

To delete (remove) data from a table, you use the DELETE statement. DELETE can be used in two ways:

- To delete specific rows from a table
- To delete all rows from a table

You'll now take a look at each of these.

Caution

Don't Omit the WHERE Clause Take special care when using DELETE because it is very easy to mistakenly delete every row from a table. Please read this entire section on DELETE before using this statement.

Tip

DELETE and Security Use of the DELETE statement can be restricted and controlled. See Chapter 28.

I stated earlier that UPDATE is very easy to use. The good (and bad) news is that DELETE is even easier to use.

The following statement deletes a single row from the customers table:

Input

```
DELETE FROM customers
WHERE cust_id = 10006;
```

This statement should be self-explanatory. DELETE FROM requires that you specify the name of the table from which the data is to be deleted. The WHERE clause filters which rows are to be deleted. In this example, only customer 10006 will be deleted. If the WHERE clause were omitted, this statement would delete every customer in the table.

DELETE takes no column names or wildcard characters. DELETE deletes entire rows, not columns. To delete specific columns, you use an UPDATE statement (as shown earlier in this chapter).

Note

Table Contents, Not Tables The DELETE statement deletes rows from tables; it can even delete all rows from a table. But DELETE never deletes the table itself.

Tip

Faster Deletes If you really do want to delete all rows from a table, don't use DELETE. Instead, use the TRUNCATE TABLE statement, which accomplishes the same thing but does it much more quickly. TRUNCATE actually drops and re-creates the table instead of deleting each row individually.

Guidelines for Updating and Deleting Data

The UPDATE and DELETE statements used in the previous sections all have WHERE clauses, and there is a very good reason for this. If you omit the WHERE clause, the UPDATE or DELETE is applied to every row in the table. In other words, if you execute an UPDATE without a WHERE clause, every row in the table is updated with the new values. Similarly, if you execute DELETE without a WHERE clause, all the contents of the table are deleted.

Here are some best practices that many SQL programmers follow:

- Never execute an UPDATE or a DELETE without a WHERE clause unless you really do intend to update or delete every row.
- Make sure every table has a primary key and use it as the WHERE clause whenever possible. You can specify individual primary keys, multiple values, or value ranges. (Refer to Chapter 15, "Joining Tables," if you have forgotten what a primary key is.)
- Before you use a WHERE clause with UPDATE or DELETE, first test it with a SELECT to make sure it is filtering the right records. It is far too easy to write incorrect WHERE clauses.
- Use database-enforced referential integrity (refer to Chapter 15 for this one, too) so MySQL will not allow the deletion of rows that have data in other tables related to them.

Caution

Use with Caution The bottom line is that MySQL has no Undo button. Be very careful when using UPDATE and DELETE, or you'll find yourself updating and deleting the wrong data.

Summary

In this chapter, you learned how to use the `UPDATE` and `DELETE` statements to manipulate the data in your tables. You learned the syntax for each of these statements, as well as the danger you face in using them. You also learned why `WHERE` clauses are so important in `UPDATE` and `DELETE` statements, and you were given guidelines to follow to help ensure that data does not get damaged inadvertently.

Challenges

1. In the United States, state name abbreviations should always be in uppercase. Write a SQL statement to update all U.S. addresses in both vendor states (`vend_state` in `Vendors`) and customer states (`cust_state` in `Customers`) so that they are uppercase. To do this, you'll need to use a function that converts text to uppercase (refer to Chapter 11, if needed) and a `WHERE` clause to filter just U.S. addresses.
2. Challenge 1 in Chapter 15 asked you to add yourself to the `Customers` table. Now delete yourself. Make sure to use a `WHERE` clause (and test it with a `SELECT` before using it in `DELETE`), or you'll delete all customers!

This page intentionally left blank